Contents

Run Book — Lumen DS: Cross-Sell & Up-Sell Recommendation System **1**
1. Prerequisites . . . 1
2. Project Structure . . . 1
3. Environment Setup . . . 2
4. Data Setup . . . 2
5. Running the Notebooks . . . 2
6. Starting the API . . . 4
7. API Endpoints . . . 4
8. Reproducing Identical Results . . . 6
9. Troubleshooting . . . 6

Run Book — Lumen DS: Cross-Sell & Up-Sell Recommendation System

Instructions for use — how to run the code to get identical results.

---

1. Prerequisites

Requirement	Version
Python	3.10
Conda	any recent version

All other dependencies are installed via the environment setup below.

---

2. Project Structure

```
lumen-DS/
├── api/
│   ├── main.py                  ← REST API (FastAPI)
│   └── requirements.txt         ← API dependencies
├── data/
│   ├── raw/
│   │   └── LUMEN_DS.csv         ← original dataset (not in git, required)
│   └── processed/               ← generated by notebooks
├── models/                      ← trained model files (generated by notebooks)
├── notebooks/
│   ├── data_cleaning.ipynb
│   ├── customer_segmentation.ipynb
│   ├── market_basket.ipynb
│   ├── cross_sell_recommender.ipynb
│   ├── upsell_model.ipynb
│   └── archive/                 ← exploratory notebooks not part of the API path
├── visualization/
└── docs/
    ├── documentation/
    │   ├── DOCUMENTATION.md         ← CRISP-DM project documentation (English)
    │   └── DOKUMENTACIJA.md         ← CRISP-DM project documentation (Croatian)
    └── runbooks/
        ├── RUN_BOOK.md              ← this file (English)
        └── TEHNIČKA_DOKUMENTACIJA.md ← Croatian version
```

---

3. Environment Setup

3.1 Create and activate the conda environment

```
conda create -n lumen-ds python=3.10 -y
conda activate lumen-ds
```

3.2 Install notebook dependencies

```
pip install pandas==2.3.3 numpy==2.2.6 scikit-learn==1.7.2 xgboost==3.2.0 \
    imbalanced-learn==0.14.1 mlxtend==0.23.4 matplotlib seaborn \
    jupyter ipykernel joblib
```

3.3 Install API dependencies

```
pip install -r api/requirements.txt
```

3.4 Register the kernel with Jupyter

```
python -m ipykernel install --user --name lumen-ds --display-name "lumen-ds"
```

---

4. Data Setup

Place the original dataset in the `data/raw/` directory:

`data/raw/LUMEN_DS.csv`

This file is not included in the repository due to its size (~795 MB). All other files in `data/processed/` and `models/` are either committed to the repository or generated by running the notebooks.

---

5. Running the Notebooks

Run the notebooks in the following order. Each notebook reads from files produced by the previous ones. Do not skip steps.

Open Jupyter from the project root:

```
conda activate lumen-ds
jupyter notebook
```

Make sure the kernel is set to **lumen-ds** before running each notebook.

Step 1 — `notebooks/data_cleaning.ipynb`

Reads: `data/raw/LUMEN_DS.csv`

Produces: `data/processed/data_clean_final.csv`

What it does: Loads the raw dataset, removes invalid rows (negative prices, zero quantities, intercompany transactions, placeholder customers and items), imputes missing values, and saves the clean dataset. This file is the input for all subsequent notebooks.

Step 2 — `notebooks/customer_segmentation.ipynb`

Reads: `data/processed/data_clean_final.csv`

Produces: `data/processed/rfm_segments.csv`

What it does: Computes RFM (Recency, Frequency, Monetary) scores for all customers using a snapshot date of 2020-07-01, then applies KMeans clustering (K=4) to produce four customer segments: Hibernating, Mid-Tier At Risk, Active Small Buyers, and Whales/Champions. The cluster labels are used as features in the cross-sell model and for personalised item ranking in both endpoints.

Step 3 — `notebooks/market_basket.ipynb`

Reads: `data/processed/data_clean_final.csv`

Produces: `data/processed/association_rules.csv`, `data/processed/basket_matrix.csv`

What it does: Prepares customer × calendar-month baskets (invoice-level baskets are too sparse at 1.8% multi-product), runs both Apriori and FP-Growth algorithms, and filters rules by lift > 1. The resulting 14 association rules are used as features in the cross-sell model.

Step 4 — `notebooks/cross_sell_recommender.ipynb`

Reads: `data/processed/data_clean_final.csv`, `data/processed/rfm_segments.csv`, `data/processed/association_rules.csv`

Produces: - `models/recommender/recommender_xgb.pkl` — trained XGBoost model - `models/recommender/model_meta.json` — model metadata and AUC scores - `data/processed/recommender/customer_feats.csv` — precomputed features for 9,017 known customers - `data/processed/recommender/customer_purchased_pgs.csv` — historical purchase history per customer - `data/processed/recommender/product_feats.csv` — features for all 30 product groups - `data/processed/recommender/copurchase_rates.csv` — directed PG-to-PG co-purchase rates - `data/processed/recommender/feature_medians.json` — population medians for cold-start customers - `data/processed/recommender/customer_recommendations.csv` — precomputed top-5 recommendations for all customers

What it does: Builds a Customer × Product Group matrix (~257k rows, 0.34% positive rate), engineers 27 features per row, trains and evaluates three models (Logistic Regression, Random Forest, XGBoost) with GroupKFold cross-validation and SMOTE oversampling, then generates cross-sell recommendations for all customers. The XGBoost model achieves ROC-AUC of **0.9248**.

Step 5 — `notebooks/upsell_model.ipynb`

Reads: `data/processed/data_clean_final.csv`, `data/processed/rfm_segments.csv`

Produces: - `data/processed/upsell/item_tiers.csv` — standard/premium classification for all items - `data/processed/upsell/pg_median_prices.csv` — median price per product group - `data/processed/upsell/upsell_item_popularity.csv` — cluster-conditional premium item rankings - `data/processed/upsell/upsell_item_popularity_all.csv` — cluster-conditional all-item rankings (used for cross-sell Layer 2) - `data/processed/upsell/upsell_recommendations.csv` — precomputed up-sell recommendations for all customers - `models/upsell/upsell_meta.json` — summary statistics

What it does: Classifies each item as standard or premium (based on whether its average price is above or below the product group median), identifies customers buying below the median in each PG, and builds cluster-conditional item popularity rankings. Validated via proxy metric: 44.2% of up-sell targets showed positive price uplift in the label period.

---

6. Starting the API

From the project root directory with the `lumen-ds` environment active:

```
conda activate lumen-ds
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
```

Expected startup output:

```
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000
```

The API loads all models and lookup tables into memory during startup (~3–5 seconds). Once running, all requests are served from memory with no disk reads.

Interactive documentation (Swagger UI) is available at:

`http://localhost:8000/docs`

This provides a browser-based interface for testing both endpoints without writing any code.

---

7. API Endpoints

`GET\/health`

Confirms the server is running and the model is loaded.

Request:

```
curl http://localhost:8000/health
```

Response:

```
{
  "status": "ok",
  "model_loaded": true,
  "known_customers": 9017,
  "layer2_available": true
}
```

`POST\/cross-sell`

Returns up to 5 product group recommendations for a customer, ordered by confidence. Each recommended product group includes up to 5 specific item suggestions.

Input fields: | Field | Type | Required | Description | |—|—|—|—| | `customer_id` | integer | No | ID of an existing customer. If omitted or unknown, population averages are used. | | `item_codes` | list of strings | No | Item codes the customer is currently ordering. Used to exclude their active PGs and compute co-purchase signals. |

Example — existing customer with current order:

```
curl -X POST http://localhost:8000/cross-sell \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 12345, "item_codes": ["03650", "2158"]}'
```

Example — new customer (no history):

```
curl -X POST http://localhost:8000/cross-sell \
  -H "Content-Type: application/json" \
  -d '{"item_codes": ["03650", "2158"]}'
```

Response:

```
{
  "customer_id": 12345,
  "recommendations": [
    {
      "product_group": "PC016",
      "confidence": 0.4642,
      "items": [
        {"item_code": "BOE001-TY8.375", "confidence": 0.1098, "avg_price": 6.02},
        {"item_code": "BOE001-348.5",   "confidence": 0.0854, "avg_price": 11.0}
      ]
    }
  ]
}
```

`confidence` on the product group is the XGBoost model probability (0–1) that the customer will purchase from that group. `confidence` on items is the fraction of similar cluster customers who bought that item.

`POST\/up-sell`

For each item code provided, checks whether it is a standard-tier item and returns premium alternatives from the same product group, personalised to the customer's cluster.

Input fields: | Field | Type | Required | Description | |—|—|—|—| | `customer_id` | integer | No | ID of an existing customer. If omitted, cluster personalisation is skipped. | | `item_codes` | list of strings | Yes | Item codes to check for up-sell opportunities. |

Example:

```
curl -X POST http://localhost:8000/up-sell \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 12345, "item_codes": ["03650", "03673"]}'
```

Response:

```
{
  "customer_id": 12345,
  "recommendations": [
    {
      "current_item": "03650",
      "product_group": "PC001",
      "current_tier": "standard",
      "current_avg_price": 0.0,
      "premium_alternatives": [
        {"item_code": "2158",          "confidence": 0.0495, "avg_price": 78.99},
        {"item_code": "BOE001-30517",  "confidence": 0.0155, "avg_price": 220.44}
      ]
    }
  ]
}
```

---

8. Reproducing Identical Results

The following settings are fixed across all notebooks to ensure reproducibility:

Setting	Value
Temporal train/test split	Train: before 2019-07-01 / Label: 2019-07-01 to 2020-07-01
Random seed	`random_state=42` on all models and SMOTE
SMOTE sampling strategy	0.1 (10% positive rate in training)
Cross-validation	GroupKFold, 5 folds, grouped by CustomerID
Top-K recommendations	5
RFM snapshot date	2020-07-01
KMeans clusters	K=4

> **Note on minor AUC variation:** XGBoost and cross-validation use `n_jobs=-1` (parallel processing). Floating-point operations in parallel are non-deterministic, which causes ±0.002 variation in AUC between runs. This does not affect the model quality or recommendations.

---

9. Troubleshooting

`FileNotFoundError:\data_clean_final.csv` Run `data_cleaning.ipynb` first. Make sure `LUMEN_DS.csv` is in `data/raw/`.

`FileNotFoundError:\recommender_xgb.pkl` Run `cross_sell_recommender.ipynb` before starting the API.

`FileNotFoundError:\association_rules.csv` Run `market_basket.ipynb` before `cross_sell_recommender.ipynb`.

API returns `"known_customers":\0` The `customer_feats.csv` inference artifact is missing. Re-run `cross_sell_recommender.ipynb` and ensure the cell that saves inference artifacts executes without error.

API returns `"layer2_available":\false` The `upsell_item_popularity_all.csv` file is missing. Re-run `upsell_model.ipynb`.

Port already in use Change the port: `uvicorn\api.main:app\-\/-port\8001`